Code-Convention

Diese Code-Convention gilt für das Java-Projekt `java_projekt`. Sie soll sicherstellen, dass der Code einheitlich, verständlich und gut wartbar bleibt.

Allgemeine Regeln

- Der gesamte Anwendungscode wird in Java geschrieben.
- Das Projekt wird mit Maven organisiert.
- Quellcode liegt unter `src/main/java`.
- Testcode liegt unter `src/test/java`.
- Klassen, Methoden und Variablen sollen sprechende Namen haben.
- Unnötiger Code, nicht verwendete Methoden und nicht verwendete Imports werden entfernt.
- Kommentare sollen erklären, warum etwas gemacht wird, nicht nur was gemacht wird.

Sprache

- Klassennamen und Methodennamen dürfen deutsch sein, wenn sie zur bestehenden Projektstruktur passen.
- Java-Schlüsselwörter und technische Begriffe bleiben englisch.
- Fehlermeldungen für Benutzer werden auf Deutsch geschrieben.
- Dateinamen müssen zum Klassennamen passen.

Beispiel:

```
public class Lernblock {
  }
```

Formatierung

- Einrückung erfolgt mit 4 Leerzeichen.
- Eine Zeile sollte möglichst kurz und gut lesbar bleiben.
- Geschweifte Klammern stehen in Java-Standardform.

Beispiel:

```
if (dauerMinuten <= 0) {
      throw new IllegalArgumentException("Die Dauer muss positiv sein.");
  }
```

Packages

Die Anwendung ist in Schichten aufgeteilt:

- `studyplanner.model`: Datenklassen wie `Modul`, `Termin`, `Lernblock`, `Erinnerung`, `Einstellung`
- `studyplanner.service`: Anwendungslogik
- `studyplanner.persistence`: Speichern und Laden
- `studyplanner.ui`: Swing-Oberfläche

Neue Klassen sollen in das passende Package einsortiert werden.

Klassen

- Jede Klasse hat genau eine klare Aufgabe.
- Klassennamen beginnen mit einem Grossbuchstaben.
- Klassen sollen nicht unnötig gross werden.
- Gemeinsame Logik soll in passende Hilfsmethoden ausgelagert werden.

Beispiel:

```
public class Termin {
  }
```

Methoden

- Methodennamen beginnen mit einem Kleinbuchstaben.
- Methodennamen sind in camelCase geschrieben
- Methodennamen beschreiben eine Aktion.
- Methoden sollen kurz und übersichtlich bleiben.
- Eine Methode soll möglichst nur eine Aufgabe haben.

Beispiele:

```
public void setTitel(String titel) {
  }

  public List<Termin> getTermine() {
  }
```

Variablen

- Variablennamen beginnen mit einem Kleinbuchstaben.
- Variablennamen sollen ihren Zweck beschreiben.
- Abkürzungen sollen vermieden werden.

Gut:

```
int dauerMinuten;
  LocalDate datum;
```

Nicht gut:

```
int d;
  LocalDate x;
```

Konstanten

- Konstanten werden mit `static final` definiert.
- Konstantennamen werden gross geschrieben.
- Wörter werden mit Unterstrich getrennt.

Beispiel:

```
private static final int MAX_DAUER_MINUTEN = 1440;
```

Validierung

- Eingaben werden so früh wie möglich validiert.
- Ungültige Werte werfen eine `IllegalArgumentException`.
- Fehlermeldungen sollen klar beschreiben, was falsch ist.

Beispiel:

```
if (name == null || name.isBlank()) {
      throw new IllegalArgumentException("Der Modulname darf nicht leer sein.");
  }
```

Javadoc

- Klassen sollen einen Javadoc-Kommentar haben.
- Öffentliche Methoden sollen einen Javadoc-Kommentar haben.
- Tests dürfen ebenfalls Javadoc enthalten, wenn dadurch klarer wird, was getestet wird.

Beispiel:

```
/**
   * Beschreibt ein Studienmodul.
   */
  public class Modul {
  }
```

Tests

- Tests werden mit JUnit 5 geschrieben.
- Testklassen liegen unter `src/test/java`.
- Testklassen enden mit `Test`.
- Tests verwenden `Assertions`.
- Tests sollen sprechende Methodennamen haben.
- Bei erwarteten Exceptions wird die Exception gespeichert und die Fehlermeldung geprüft.

Beispiel:

```
IllegalArgumentException exception = Assertions.assertThrows(
          IllegalArgumentException.class,
          () -> new Modul("", "#123456", "", 1)
  );

  Assertions.assertEquals("Der Modulname darf nicht leer sein.", exception.getMessage
  ());
```

Maven

- Abhängigkeiten werden nur in der `pom.xml` eingetragen.
- Build-Ordner wie `target/` werden nicht ins Git-Repository aufgenommen.
- Tests werden mit folgendem Befehl ausgeführt:

```
mvn test
```

Git

- Nur relevante Quellcode-, Test- und Dokumentationsdateien werden versioniert.
- Automatisch erzeugte Dateien wie `target/` oder `out/` werden ignoriert.
- Vor dem Abgeben sollte `mvn test` erfolgreich laufen.

Clean Code

- Doppelte Logik soll vermieden werden.
- Unbenutzte Imports, Variablen und Methoden werden entfernt.
- Code soll lieber einfach und klar sein als unnötig komplex.
- Änderungen sollen zur bestehenden Projektstruktur passen.